

SiliQun: A Standards-Compliant, Pulse-Level Simulation Platform for Silicon Spin Qubits

Abstract

Silicon spin qubits represent a highly promising platform for scalable quantum computing, benefiting from advanced semiconductor manufacturing processes. However, the development of robust control protocols and quantum algorithms for these devices requires simulation tools that accurately model their unique physics, particularly at the pulse level where exchange interactions and charge noise dominate. We present SiliQun v2.0, an open-source, standards-compliant simulation platform tailored for silicon spin qubits. SiliQun bridges the gap between hardware physics and high-level quantum software by integrating a Lindblad master equation solver for accurate time-domain evolution with widely adopted quantum computing standards. The platform natively supports OpenQASM 3.0 for circuit interchange, OpenPulse for schedule representation, and provides direct interoperability with the Qiskit and PennyLane ecosystems. We demonstrate SiliQun's capabilities through process tomography of exchange-driven two-qubit gates and fidelity forecasting under realistic noise profiles. By providing a seamless interface from quantum machine learning workflows to hardware-accurate pulse simulations, SiliQun accelerates the co-design of algorithms and control strategies for next-generation silicon quantum processors.

1. Introduction

The pursuit of fault-tolerant quantum computing requires hardware platforms capable of supporting millions of qubits. Silicon spin qubits [1] have emerged as a leading candidate for large-scale integration due to their long coherence times, compact footprint, and compatibility with established complementary metal-oxide-semiconductor (CMOS) manufacturing facilities [2]. Recent experimental breakthroughs have demonstrated high-fidelity single- and two-qubit gates [3], multi-qubit entanglement, and the operation of linear arrays and 2D grids of quantum dots [4].

Despite these hardware advances, the software ecosystem for silicon spin qubits remains fragmented. Algorithm developers typically rely on abstract circuit simulators that assume idealized, instantaneous gates, while experimentalists utilize low-level physics solvers that are disconnected from modern quantum software development kits (SDKs). This disconnect impedes the co-design process: algorithms optimized for abstract gate sets often perform poorly when translated to the native exchange interactions of silicon devices, and hardware-specific control strategies are difficult to integrate into higher-level workflows such as quantum machine learning (QML) or variational quantum eigensolvers (VQE).

To address this challenge, we introduce SiliQun v2.0, a generalized, standards-compliant simulation platform designed specifically for silicon spin qubits. Building upon its predecessor, which focused primarily on deep reinforcement learning (DRL) for quantum control, SiliQun v2.0 provides a comprehensive infrastructure that spans from abstract circuit definitions to noise-accurate pulse-level execution.

The core contributions of this work are:

- 1. Pulse-Level Simulation Engine:** A high-performance Lindblad master equation solver that accurately models the time-domain evolution of spin qubits under drive and exchange pulses, incorporating realistic noise models such as $1/f$ charge noise and relaxation.
- 2. Standards Integration:** Native support for OpenQASM 3.0 [5] and OpenPulse [6], ensuring compatibility with the broader quantum software ecosystem and facilitating the exchange of circuits and pulse schedules.
- 3. Ecosystem Interoperability:** Direct plugin interfaces for Qiskit (Backendv2) [7] and PennyLane (Device API) [8], enabling seamless execution of QML and hybrid quantum-classical algorithms on simulated silicon hardware.

4. **Comprehensive Toolchain:** Integrated modules for gate-to-pulse compilation, state and process tomography, and fidelity forecasting.

This paper is structured as follows. Section 2 details the architecture of SiliQun and its integration with quantum computing standards. Section 3 describes the pulse-level simulation engine and the underlying physical models. Section 4 discusses interoperability with PennyLane and Qiskit. Section 5 presents experimental benchmarks, including process tomography of native gates. Finally, Section 6 discusses future directions, and Section 7 concludes.

2. Architecture and Standards Integration

The architecture of SiliQun v2.0 is designed to provide a modular, extensible framework that bridges high-level quantum programming and low-level physical simulation. The platform is organized into four distinct layers, each adhering to widely adopted industry standards.

2.1 Layered Architecture

1. **API and Ecosystem Layer:** This top layer provides the interface to external users and frameworks. It includes a FastAPI-based REST server for remote job execution and native plugins for Qiskit and PennyLane.
2. **Compilation Layer:** This layer translates abstract quantum circuits into hardware-specific representations. It features an OpenQASM 3.0 parser and a gate-to-pulse compiler that maps standard logic gates (e.g., CNOT, CZ) to sequences of native exchange interactions and single-qubit microwave drives.
3. **Pulse Control Layer:** Operating at the level of analog control signals, this layer manages the scheduling and representation of pulses. It utilizes the OpenPulse grammar to define drive and exchange channels, ensuring that pulse schedules can be exported to or imported from compatible hardware controllers.
4. **Physics Simulation Layer:** The foundational layer consists of the numerical solvers that compute the time evolution of the quantum state. It includes the Lindblad master equation solver for pulse-level simulation and tensor-network backends (MPS/MPO) for scalable circuit-level execution.

2.2 Standards Adoption

To maximize utility and interoperability, SiliQun adopts the following standards:

- **OpenQASM 3.0:** Chosen as the primary circuit interchange format, OpenQASM 3.0 provides a robust grammar for defining quantum circuits, including classical control flow and timing constraints. SiliQun's compiler natively parses OpenQASM 3.0 strings, automatically injecting standard gate definitions (`stdgates.inc`) and mapping them to the silicon spin qubit native gate set.
- **OpenPulse:** The OpenPulse specification provides a standardized method for describing microwave and baseband control signals. SiliQun implements OpenPulse-compatible schedule representations, allowing researchers to define custom pulse shapes (e.g., Gaussian, square) and apply them to specific device channels (drive, exchange, measure).
- **Qiskit BackendV2:** By implementing the `BackendV2` interface, SiliQun acts as a drop-in replacement for IBM Quantum hardware within the Qiskit ecosystem. This allows users to leverage Qiskit's powerful transpilation pipeline and noise mitigation tools before executing circuits on the SiliQun simulator.

3. Pulse-Level Simulation with Lindblad Master Equation

The defining feature of SiliQun v2.0 is its ability to simulate quantum operations at the pulse level, capturing the continuous-time dynamics of spin qubits under realistic noise conditions.

3.1 Hamiltonian and Control

For an array of silicon spin qubits, the system Hamiltonian $H(t)$ is composed of a static Zeeman splitting term and time-dependent control terms: $H(t) = \sum_i \frac{\hbar \nu_i}{2} \sigma_z^{(i)} + \sum_i \Omega_i(t) \cos(2\pi \nu_i t + \phi_i) \sigma_x^{(i)} + \sum_{\langle i,j \rangle} J_{ij}(t) (\sigma_x^{(i)} \sigma_x^{(j)} + \sigma_y^{(i)} \sigma_y^{(j)} + \sigma_z^{(i)} \sigma_z^{(j)})$ where ν_i is the Larmor frequency of qubit i , $\Omega_i(t)$ is the microwave drive amplitude, and $J_{ij}(t)$ is the tunable exchange interaction strength between adjacent qubits. SiliQun's `PulseSequence` object allows users to define $\Omega_i(t)$ and $J_{ij}(t)$ using standard waveform shapes.

3.2 Noise Modeling

Environmental noise is incorporated via the Lindblad master equation for the density matrix $\rho(t)$: $\frac{d\rho}{dt} = -\frac{i}{\hbar} [H(t), \rho] + \sum_k \left(L_k \rho L_k^\dagger - \frac{1}{2} \{L_k^\dagger L_k, \rho\} \right)$ SiliQun natively supports the primary decoherence mechanisms in silicon:

- **Relaxation (T_1):** Modeled using the lowering operator $L_1 = \sqrt{1/T_1} \sigma_-$.
- **Dephasing (T_2^*):** Modeled using the phase operator $L_2 = \sqrt{1/(2T_2^*)} \sigma_z$.
- **Charge Noise:** Low-frequency $1/f$ charge noise, which strongly affects the exchange interaction J_{ij} , is simulated via quasi-static Hamiltonian variations averaged over multiple trajectories, or perturbatively integrated into the dephasing operators.

The solver utilizes adaptive Runge-Kutta integration methods provided by the SciPy ecosystem, ensuring numerical stability and accuracy even for complex, overlapping pulse sequences.

4. Ecosystem Interoperability

To ensure SiliQun is immediately useful to algorithm developers, we implemented deep integrations with two leading quantum software frameworks: PennyLane and Qiskit.

4.1 PennyLane Device Plugin

PennyLane [8] is the standard framework for quantum machine learning and differentiable quantum programming. We implemented `SiliQunDevice`, a custom plugin conforming to PennyLane's `Device` API v2. This integration allows users to decorate Python functions with `@qml.qnode(dev)` and execute them directly on the SiliQun simulation engine. The plugin supports state vector extraction, expectation value measurement (`qml.expval`), and probability sampling. Crucially, this enables the training of variational quantum circuits where the forward pass is evaluated using SiliQun's noise-accurate physics engine, providing a more realistic cost landscape than idealized simulators.

4.2 Qiskit Backend Interface

Qiskit [7] provides a comprehensive suite of tools for circuit construction, transpilation, and error mitigation. By exposing `SiliQunBackendV2`, the platform integrates seamlessly into the Qiskit transpiler pipeline. Users can submit Qiskit `QuantumCircuit` objects directly to the backend. The transpiler automatically decomposes abstract gates into the supported basis set (`rx`, `ry`, `rz`, `cz`) and applies topology-aware routing based on the selected device profile (e.g., 1D chain for SiMOS, 2D grid for SLEDGE).

5. Experiments and Benchmarks

To validate the generalized platform, we conducted a series of benchmarks focusing on the accuracy of the gate-to-pulse compiler and the fidelity of the resulting operations under noise.

5.1 Gate-to-Pulse Compilation and Tomography

We evaluated the compilation of a CNOT gate, which is not native to the exchange-driven architecture. The compiler decomposes the CNOT into a sequence involving a native CZ gate (implemented via a precisely timed exchange pulse) and single-qubit rotations.

Using SiliQun’s built-in tomography module, we performed quantum process tomography (QPT) on the compiled CNOT gate. In the absence of noise, the maximum likelihood estimation (MLE) reconstructed process matrix exhibited a fidelity of > 0.999 compared to the ideal CNOT unitary, confirming the correctness of the pulse sequence and the integration of the Lindblad solver.

5.2 Fidelity Forecasting

We utilized the integrated Q-Forge fidelity forecasting tool to estimate the performance of a 4-qubit GHZ state preparation circuit under realistic SiMOS noise parameters ($T_1 = 2$ ms, $T_2^* = 20$ μ s). The forecast predicted a state fidelity of 0.92, closely matching the results obtained from full density matrix simulation using the Lindblad solver. This validates the utility of SiliQun for rapid, accurate performance estimation prior to full pulse-level simulation.

6. Discussion and Future Work

The release of SiliQun v2.0 establishes a robust, standards-compliant foundation for silicon spin qubit research. By decoupling the simulation engine from its original DRL-specific wrapper, we have broadened its applicability to the entire quantum software stack.

Future development will focus on three key areas:

1. **Advanced Noise Modeling:** Implementing non-Markovian noise solvers and correlated charge noise models to capture the complex low-frequency dynamics typical of semiconductor environments.
2. **Modular Architectures:** Extending the device profiles to support distributed, modular quantum computing architectures, such as networks of 6-qubit modules connected via coherent quantum links.
3. **Hardware Integration:** Developing interfaces to export OpenPulse schedules directly to arbitrary waveform generators (AWGs) for execution on physical silicon quantum dot devices.

7. Conclusion

SiliQun v2.0 provides a vital link between abstract quantum algorithms and the physical realities of silicon spin qubits. By integrating a highly accurate pulse-level simulation engine with industry standards like OpenQASM 3.0 and OpenPulse, and offering seamless interoperability with Qiskit and PennyLane, SiliQun empowers researchers to co-design the next generation of quantum hardware and software.

References

- [1] Loss, D., & DiVincenzo, D. P. (1998). Quantum computation with quantum dots. *Physical Review A*, 57(1), 120. [2] Vandersypen, L. M., et al. (2017). Interfacing spin qubits in quantum dots and donors—hot, dense, and coherent. *npj Quantum Information*, 3(1), 34. [3] Xue, X., et al. (2022). Quantum logic with spin qubits crossing the surface code threshold. *Nature*, 601(7893), 343-347. [4] Philips, S. G., et al. (2022). Universal control of a six-qubit quantum processor in silicon. *Nature*, 609(7929), 919-924. [5] Cross, A. W., et al. (2022). OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3(3), 1-50. [6] McKay, D. C., et al. (2018). Qiskit backend specifications for OpenQASM and OpenPulse experiments. *arXiv preprint arXiv:1809.03452*. [7] Qiskit contributors. (2023). Qiskit: An open-source framework for quantum computing. [8] Bergholm, V., et al. (2018). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*.